Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 1

Introduction

Welcome to CS 213

Course Logistics

- Every week
  - Three lectures
  - One lab
  - Exercises for practice
- Learning Management System: Bodhitree
  - Lab submissions
  - Slides will be available soon after/before lecture
- Evaluation plan (subject to change)
  - Evaluation for CS 213:
    - Participation (10%)
    - Two quizzes (15% x 2)
    - Two exams (25% + 35%)
  - Evaluation for CS 293:
    - Weekly lab evaluation (20%)
    - Two lab quizzes (15% x 2)
    - Two exams (20% + 30%)
- More logistics in the coming days!

Academic Integrity

Do not copy, consult or share

in quizzes, exams and evaluated lab submissions!

Do not copy or use LLMs in evaluated lab submissions.
You can consult references provided, and ask TAs for guidance/hints.
But blindly copying defeats the purpose of the exercises.

All code submitted should be written by yourself

Any suspected cheating will be reported to D-ADAC

What is in store in CS 213/293?

Data Structures and (related) Algorithms

- In 2017, Instagram switched from "BTrees" to "LSM Trees", for storing activity feed
- What are these "trees"? And why did they matter?
  - BTrees data structure, invented in 1970, is a gold standard in databases. (We will see this later in the course.)
  - LSM Trees (invented in 1996) is specialised for exploiting higher sequential write-speeds available in hardware. Turned out to be an important consideration at Instagram's subsequent scale.

Data Structures and (related) Algorithms

- In almost every program dealing with large amounts of data, a careful choice of data structures is crucial for efficiency
  - Many "standard" data structures are available as part of libraries
- In this course:
  - What, why and how of many standard data structures
    - What: lists, queues, trees, hash tables, ...
      Why: Their efficiency features/limitations
      How: Their internal workings
  - Some algorithms for data manipulation (sorting, compression, ...)

A Quick CS 101 Recap

A C++ Program

```
#include <iostream>
int main() {
  std::cout << "Hello Data Structures!" << std::endl;
  return 0;
}
```

Another C++ Program


```
#include <iostream>
struct node {
  std::string word;
  node* next = nullptr;
  void print() {
    std::cout << word << " ";
    if(next) next->print();
    else std::cout << std::endl;
  }
};
```

```
int main() {
  node* first = new node{"Hello"};
  first->next = new node{"World!"};
  first->print();
  return 0;
}
```

Let's Get Started

With a Little Experiment

Store and Retrieve

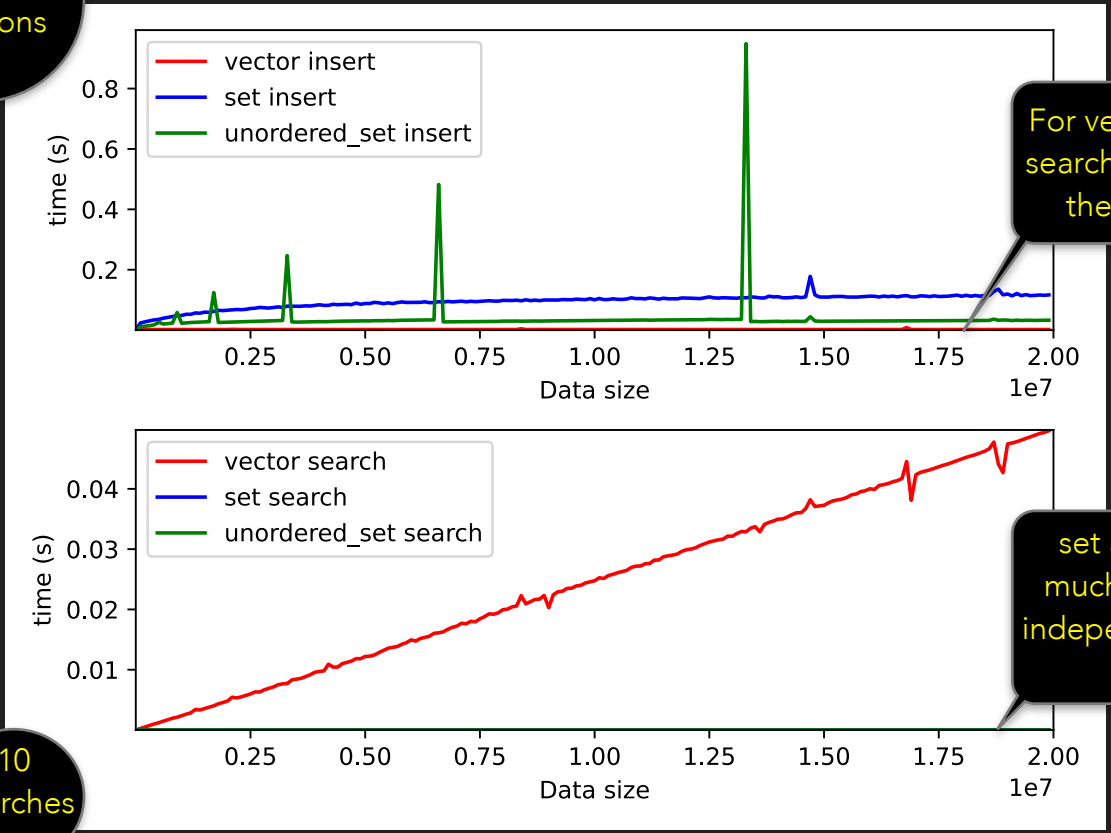
- Task: Store a bunch of integers in a data structure, and later given a query, check if it was already stored
- C++ provides convenient data structures for this, with a "find" method
  - std::vector
  - std::set
  - std::unordered_set
- Let's find out how they each perform

Store and Retrieve

```
std::vector<int> v;
std::set<int> s;
std::unordered_set<int> u;
...
// in a loop
  v.push_back(item);
  s.insert(item);
  u.insert(item);
...
std::find(v.begin(), v.end(), query); //iterate through the range
s.find(query);
u.find(query);
```

Store and Retrieve

100000 insertions



For vector, insert is very fast, but search gets slower and slower as the number of items grows

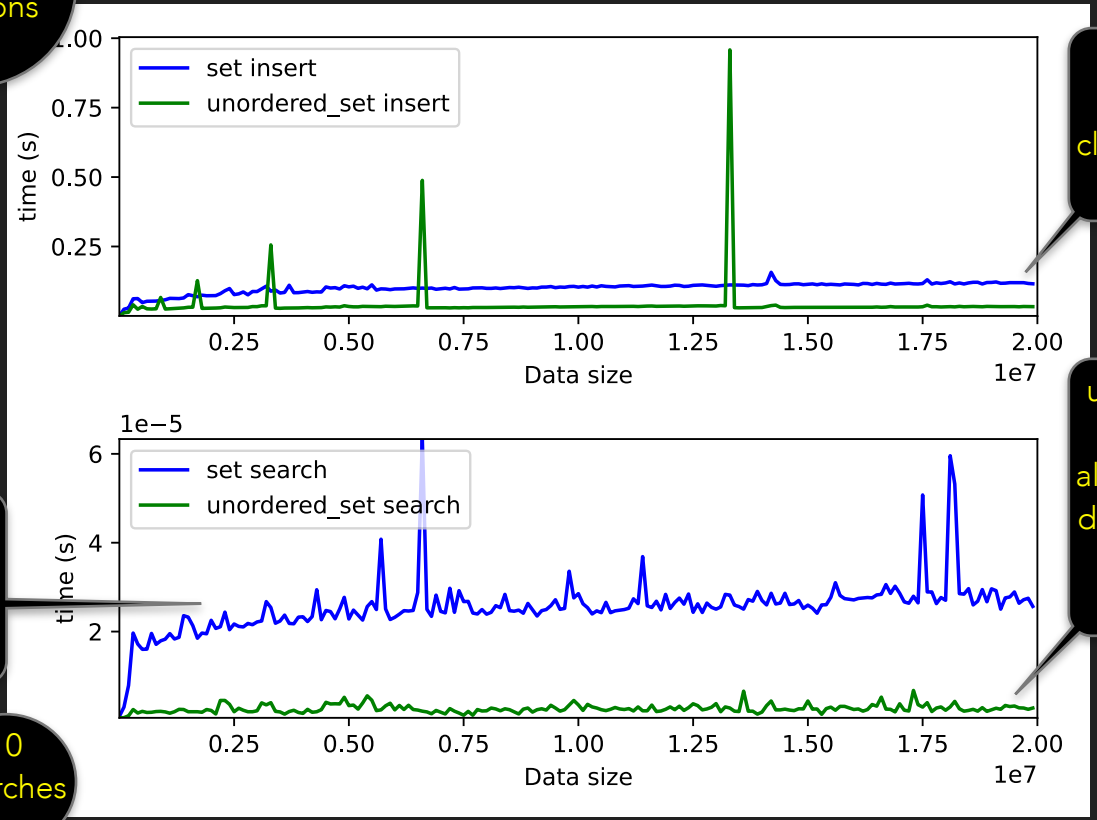
set and unordered_set have much faster search, seemingly independent of how big the data set is

10 searches

Store and Retrieve

100000 insertions

A closer look at set vs. unordered_set



set insert is also slowly climbing with the data size

unordered_set insert and search both take time almost independent of the data-size (except for insert times spiking at various data sizes)

set search is slowly climbing with the data size

10 searches

Data Structures and (related) Algorithms

- What is under the hood of std::set and std::unordered_set, and why do they behave so differently from each other?
- Looking ahead:
  - std::set is a “balanced binary search tree”
  - std::unordered_set is a “hash table”

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 2

Asymptotics

Measuring the Trends

Course Logistics

- For the Lab today, find out group you are assigned to at https://bit.ly/A26-DSA-Mapping (needs IITB Google Suite login)

Recall Our Experiment

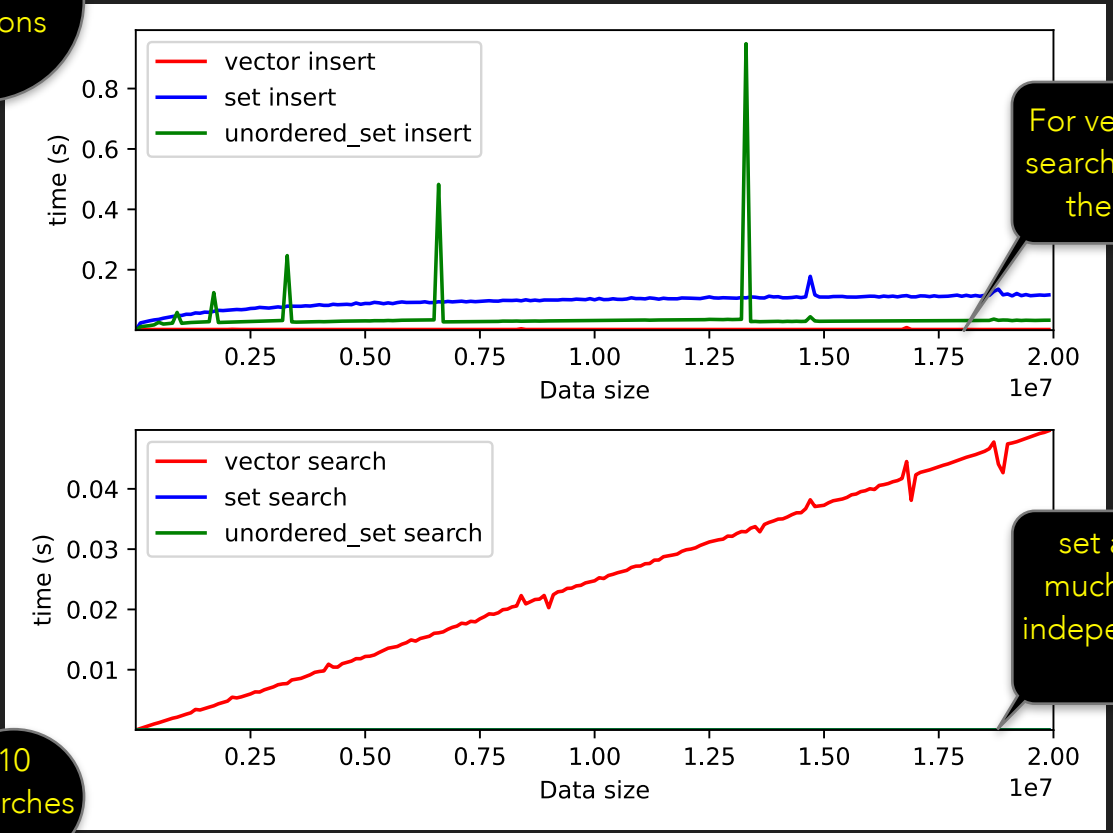
Store and Retrieve


```
std::vector<int> v;
std::set<int> s;
std::unordered_set<int> u;
...
// in a loop
  v.push_back(item);
  s.insert(item);
  u.insert(item);
...
std::find(v.begin(), v.end(), query); //iterate through the range
s.find(query);
u.find(query);
```

Store and Retrieve

Recall

100000 insertions



For vector, insert is very fast, but search gets slower and slower as the number of items grows

set and unordered_set have much faster search, seemingly independent of how big the data set is

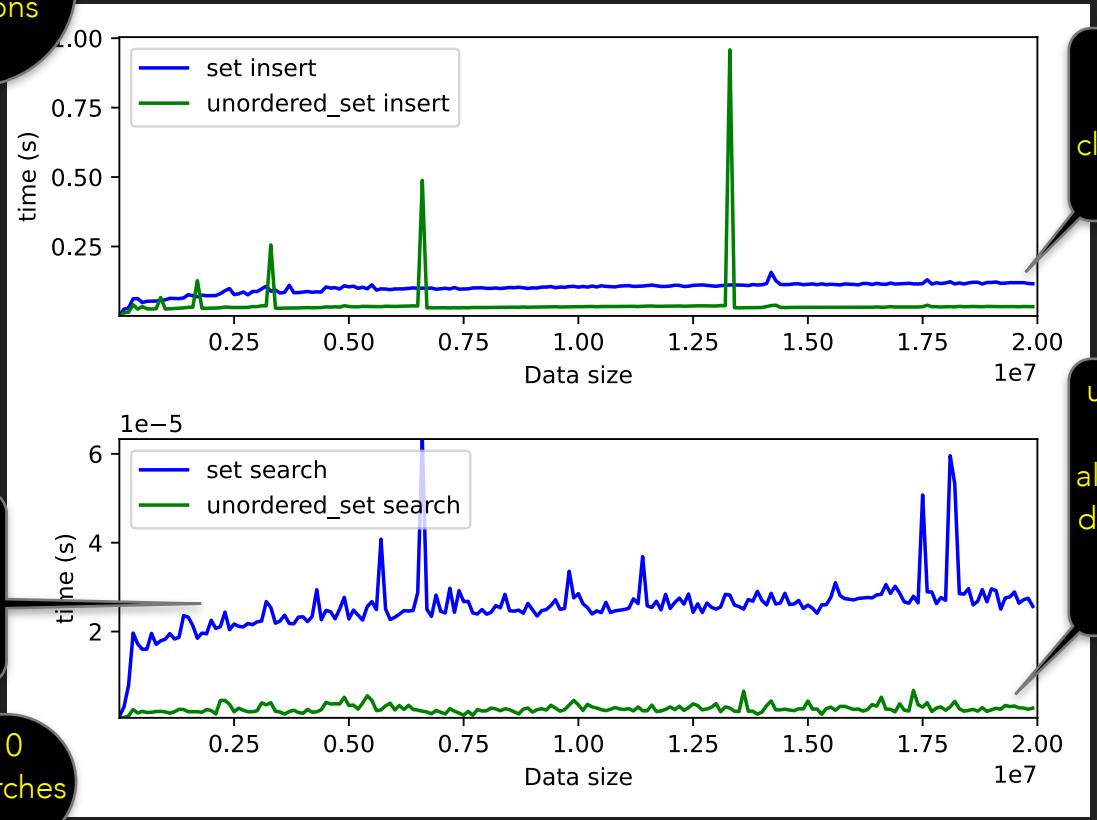
10 searches

Store and Retrieve

Recall

100000 insertions

A closer look at set vs. unordered_set



set insert is also slowly climbing with the data size

unordered_set insert and search both take time almost independent of the data-size (except for insert times spiking at various data sizes)

set search is slowly climbing with the data size

10 searches

Measuring Behaviour

- What is under the hood of std::set and std::unordered_set, and why do they behave so differently from each other?
- But first: we need some vocabulary to discuss their behaviour
  - Asymptotic Worst-Case upper bounds
- Let us unpack that

Asymptotic Behaviour

Asymptotic Behaviour

- Imagine a quantity, that starts growing starting from 1 item on day 0, in one of three ways
  - Each day, increases by a million: 1, 1000001, 2000001, ...
  - On day n, increases by 10000n: 1, 10001, 30001, 60001, ...
  - Each day, doubles: 1, 2, 4, 8, ...

$A(n) = 10^6 n + 1$

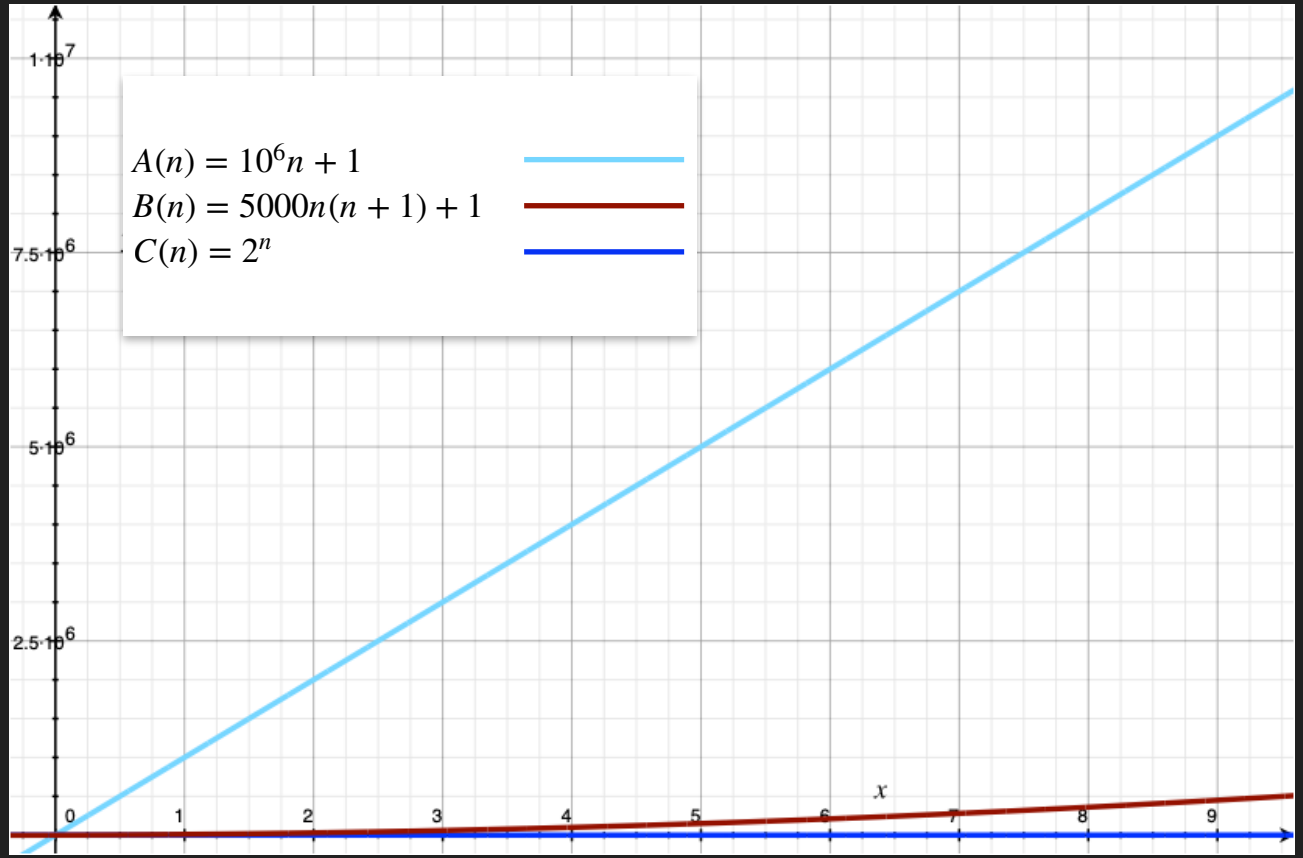
$B(n) = 10000n(n+1)/2 + 1$

$C(n) = 2^n$

- After a week, which rule will result in the largest amount? After a month?

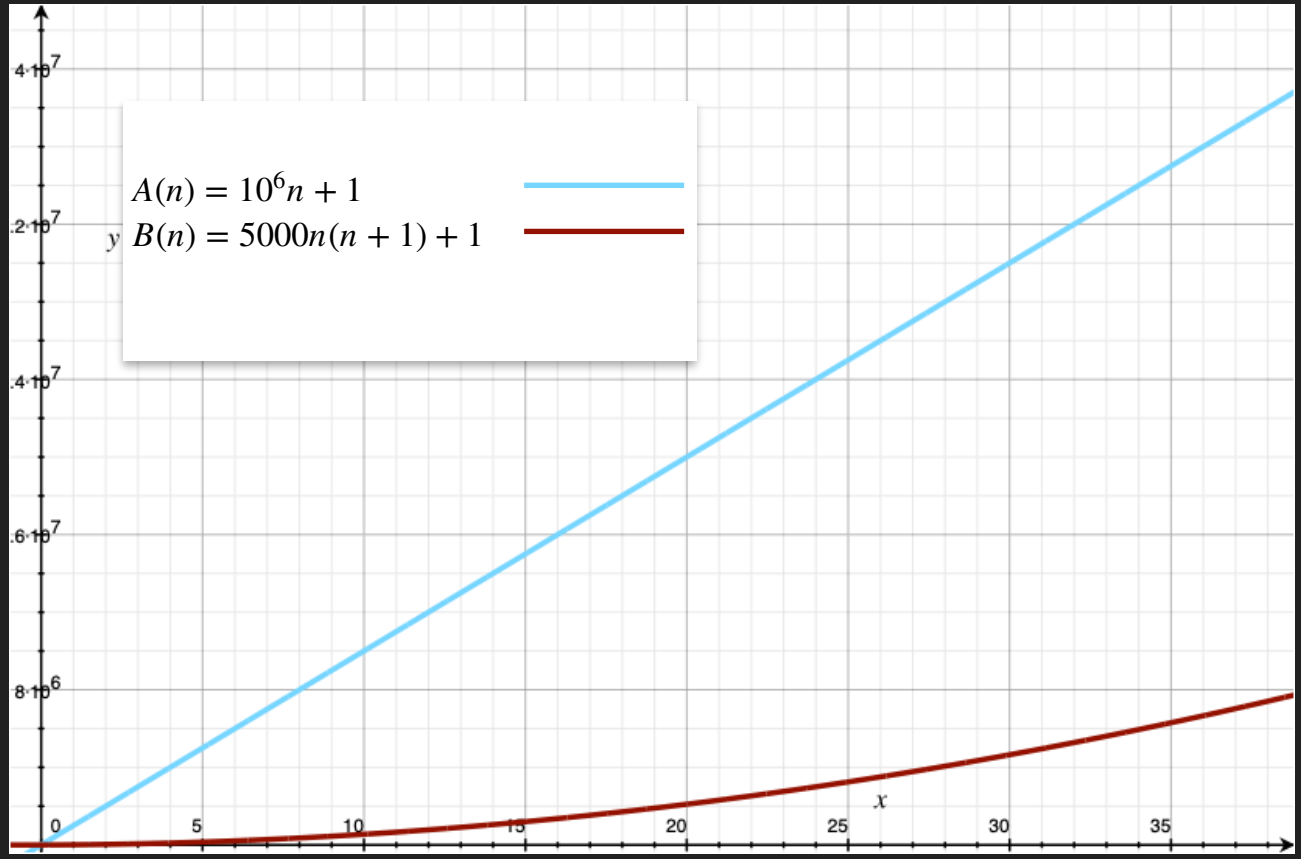
Asymptotic Behaviour

The first couple of weeks



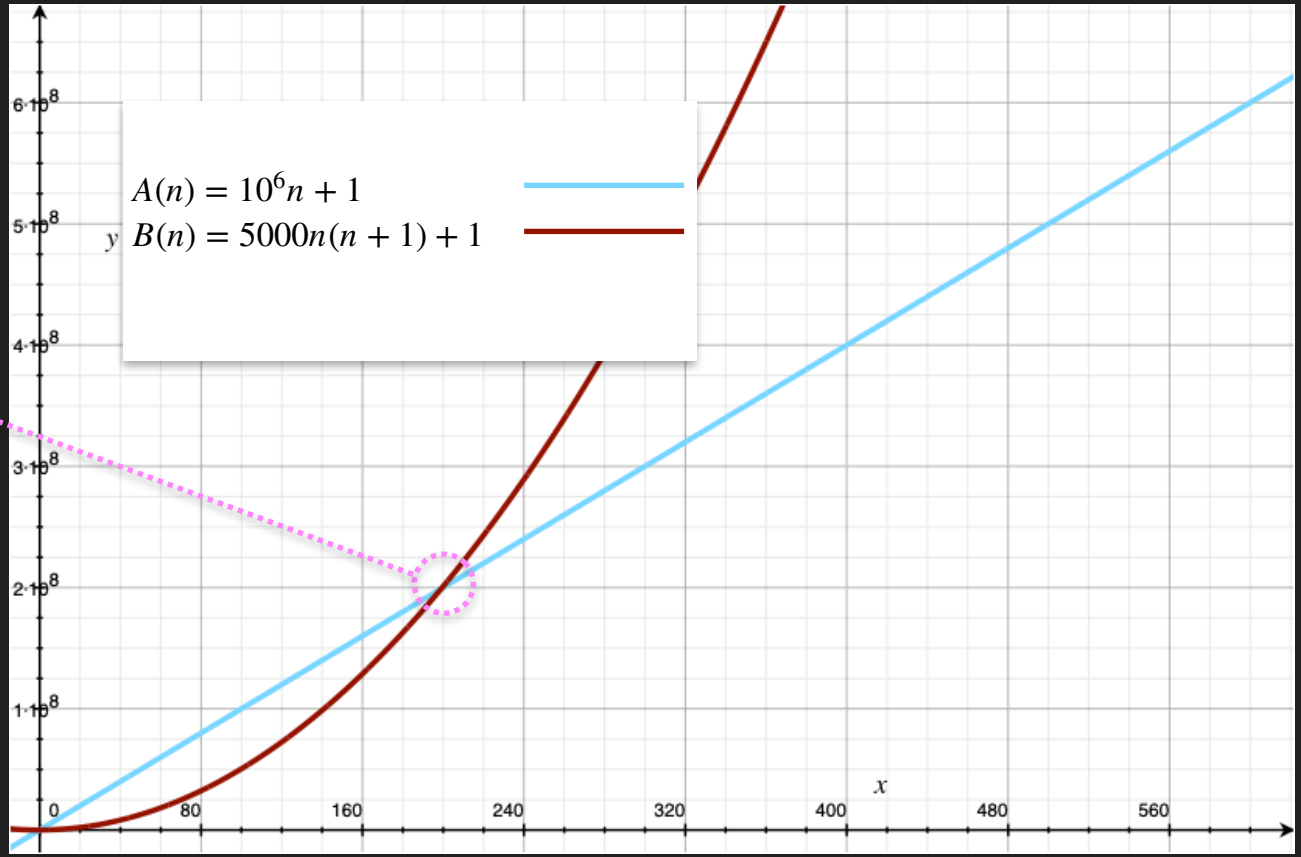
Asymptotic Behaviour

A vs. B:
First month



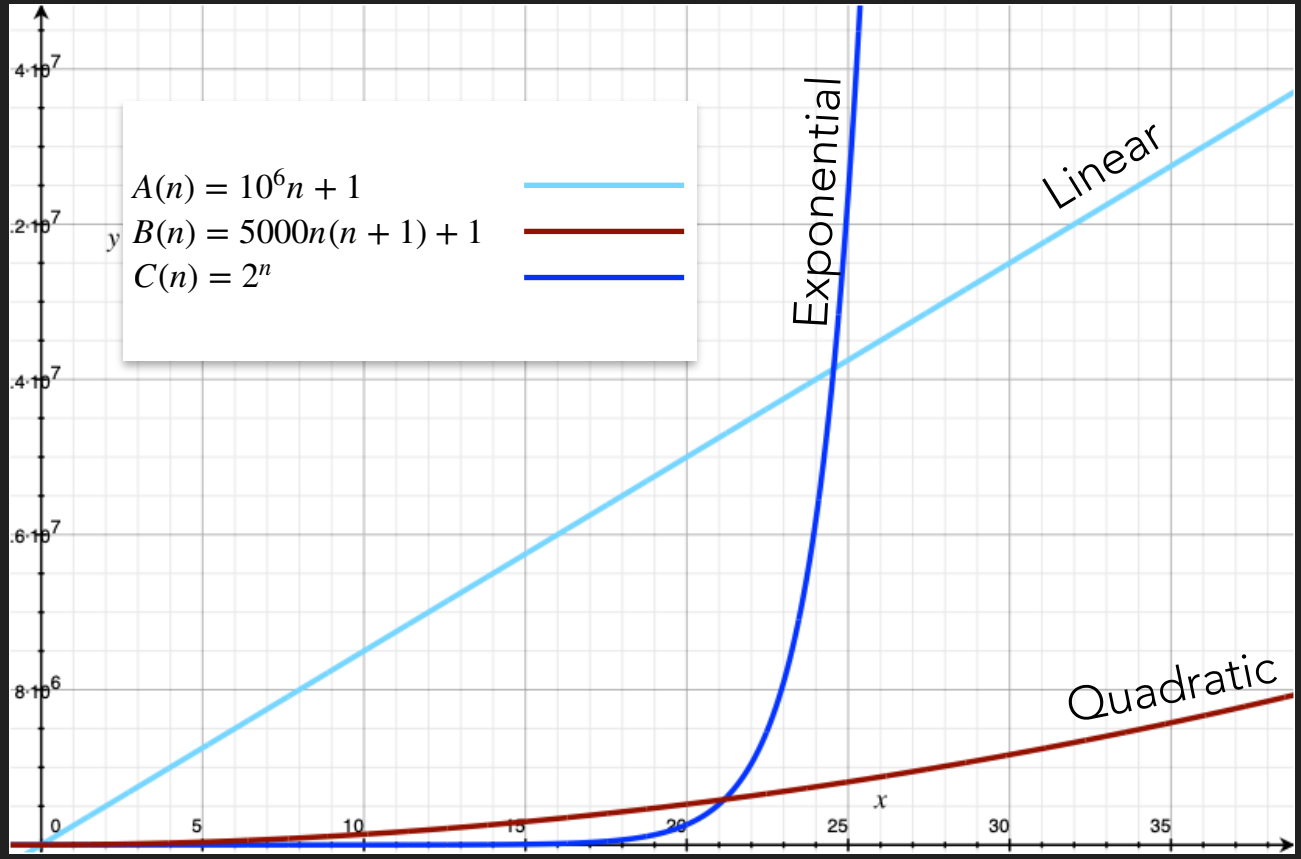
Asymptotic Behaviour

A vs. B:
200 days later



Asymptotic Behaviour

Back to the first month



Eventually,

Exponential
>> Quadratic
>> Linear

Asymptotic Behaviour

- *We say* f(n) has an asymptotically linear upper bound, if there is a scaling factor $c > 0$ and a threshold $k > 0$, such that for all $n > k$, $f(n) \leq c.n$
  - The "Big-Oh" notation: $f(n) = O(n)$
- And a quadratic upper bound, or $f(n) = O(n^2)$, if $\exists c, k$ s.t. $\forall\ n>k$, $f(n) \leq c.n^2$
- More generally, $f(n) = O(g(n))$ if $\exists c, k$ s.t. $\forall\ n>k$, $f(n) \leq c.g(n)$
- E.g., Suppose $f(n) = 5n^2 + 2n + 1$.
  - Then $f(n) = O(n^2)$. $5n^2 + 2n + 1 \leq 6n^2$ once $2n+1 \leq n^2$, or $n \geq 3$
  - But f(n) is not O(n).

If $5n^2 + 2n + 1 \leq cn$ for some c, then $n^2 \leq cn$. But this holds only for $n \leq c$, and not all $n \geq$ any k.

Asymptotic Behaviour

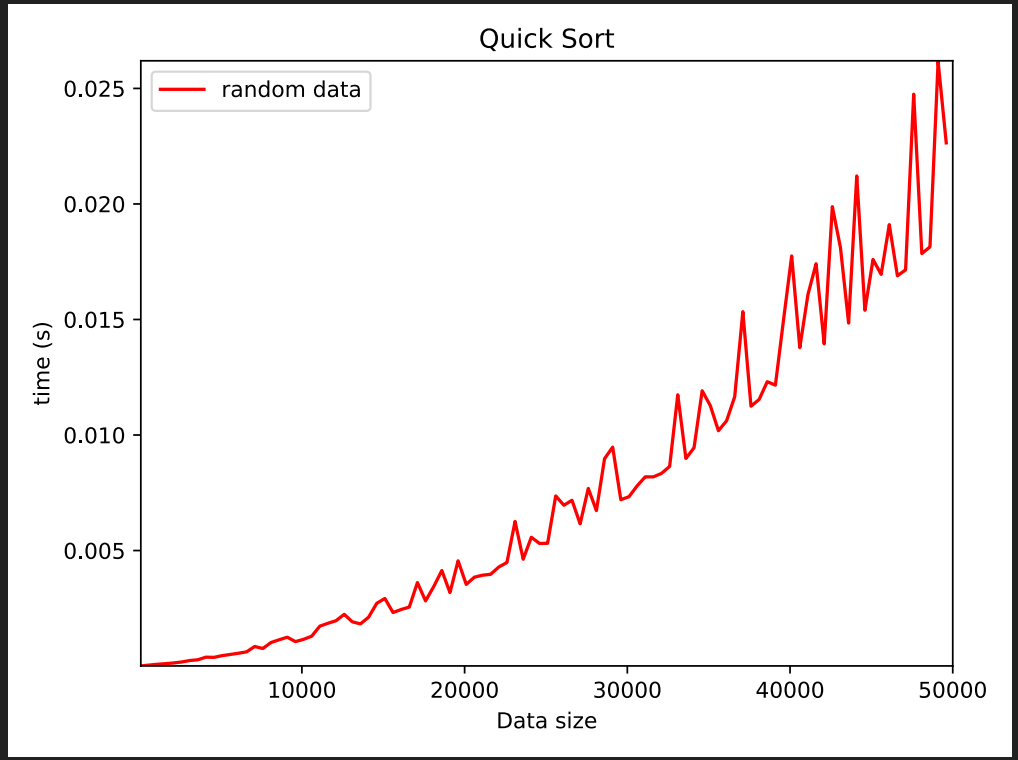
- We are typically interested in the question of scaling (to very large inputs)
  - E.g., If processing a query involving million data points takes 1s, how long will it take to process one involving 10 million data points?
    - Will it take about the same, 10x more, or 100x more, or something else?
    - Depends on how the processing algorithm scales.
    - If processing time (as a function of input data size n) is O(n), then roughly 10 seconds. If it is $O(n^2)$, roughly 100 seconds.

Asymptotic Behaviour

- $O(\cdot)$ is an upper bound
  - E.g., $f(n) := 5n^2 + 2n + 1 = O(n^2)$. But it is also $O(n^3)$, $O(n^4)$ etc.
  - If $f(n) = O(g(n))$ and $g(n)=O(f(n))$, we write $f(n) = \Theta(g(n))$
    - Equivalently, $f(n) = \Theta(g(n))$ iff there exist positive constants α, β, k s.t. for all $n \geq k$, $\alpha.g(n) \leq f(n) \leq \beta.g(n)$
- But what would f be?
  - Running time of an algorithm (in some abstract machine). But on which input? Worst-Case!

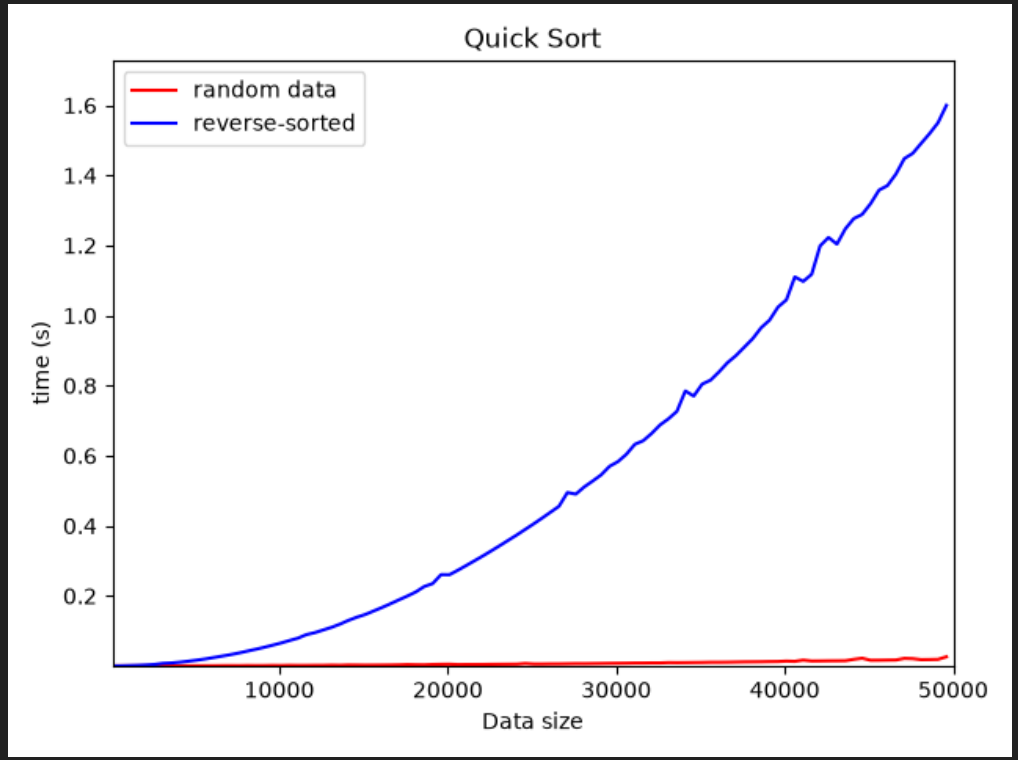
An Experiment

- QuickSort is a popular sorting algorithm, the basic form for which we'll experiment with
- On random data
  - Not O(n)
  - Turns out to be Θ(n log n)



An Experiment

- QuickSort is a popular sorting algorithm, the basic form for which we'll experiment with
- On random data
  - Not O(n)
  - Turns out to be Θ(n log n)
- On reverse sorted data, takes *much* longer
  - $\Theta(n^2)$



Asymptotic Worst-Case Time

- Asymptotic: What is the trend as data grows
- Worst-Case: Algorithms can perform wildly differently on different kinds of inputs. We should ideally be prepared for the worst-case.
  - Calculate O(.) upper bounds or Θ(.) bounds *analytically.*
- Some examples we will encounter: O(1), O(log n), O(n), O(n log n), $O(n^2)$
  - std::set has Θ(log n) insertion and search (where n is the current set size)
  - std::unordered_set has Θ(1) search; insert can occasionally be Θ(n).
  - std::vector has Θ(1) insert and Θ(n) search.

Data-Structures & Algorithms

with

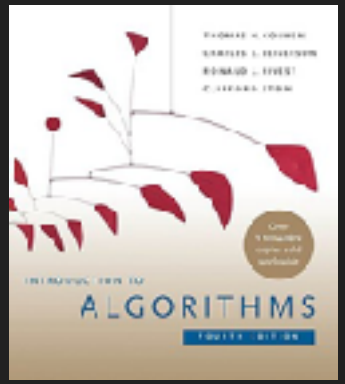
Manoj Prabhakaran

Lecture 3

Linked Lists

Course Logistics

- Bodhi Tree
  - We'll use this as a portal for all course-related activities
  - Discussion forum, announcements, labs, lecture slides
- Reference textbook: CLRS
  - We don't need everything in the book
  - And we'll cover a few things not in the book



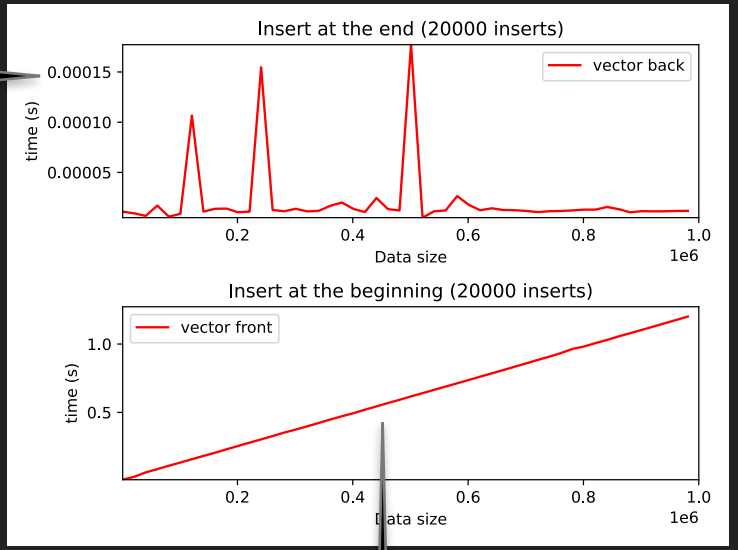
Arrays

- We saw std::vector has Θ(1) insertion (and Θ(n) search) time
- But that is only if you insert at the back of the vector
  - In std::vector, `v.push_back(x)`
- What happens if you insert elsewhere?
- A vector uses a contiguous array of memory locations to store its elements
- To insert at location i, you need to physically move the memory contents at locations i, i+1, i+2, … to make room for the new element
  - O(n-i) time, if n is the size of the vector
  - Can we really see this?

Inserting into an Array

```
std::vector<int> v1, v2;
...
// in a loop
  ...
  v1.insert(v1.begin(), 1); // add as v[0]
  v2.insert(v2.end(), 1); // v2.push_back(1)
...
```

microseconds vs 1 second!



Θ(1) vs Θ(n)

An Alternative to Arrays

- Linked lists: Maintain an ordering of the items, while still allowing O(1) insertion to the front and the back
- A list of “nodes” which do not have to be in contiguous memory locations in order
- Each node has a pointer to the next one in the list
  - The last node has `nullptr` as next

O(1) insertion

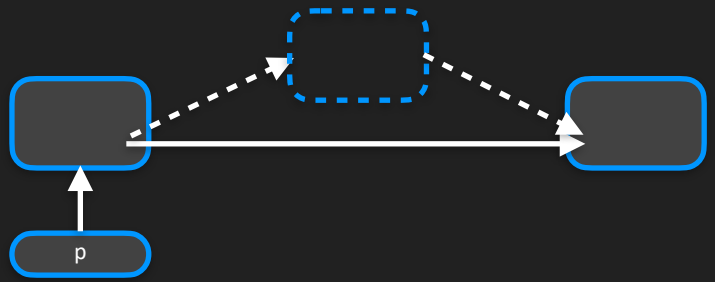
```
struct node { int val; node* next = nullptr; }; // a node in the list
int main() {
  node *head, *tail;
  head = tail = new node{0}; //a list with a single node
  // insert at the end
  tail->next = new node{1}; //attach a new node at the back
  tail = tail->next;        //point tail to the new node
  // insert at the beginning
  head = new node{-1,head};
}
```

```
node* tmp = new node;
tmp->val = -1;
tmp->next = head;
head = tmp;
```

O(1) insertion

- Can insert not just to the front and back, but anywhere in a list
  - Once we have a position in the list to insert into, insertion itself takes a fixed number of operations, irrespective of the size of the list

```
// p points to a node; to insert a new node with val x after that node
p->next = new node{x,p->next};
if(p==end) end = p->next;
```

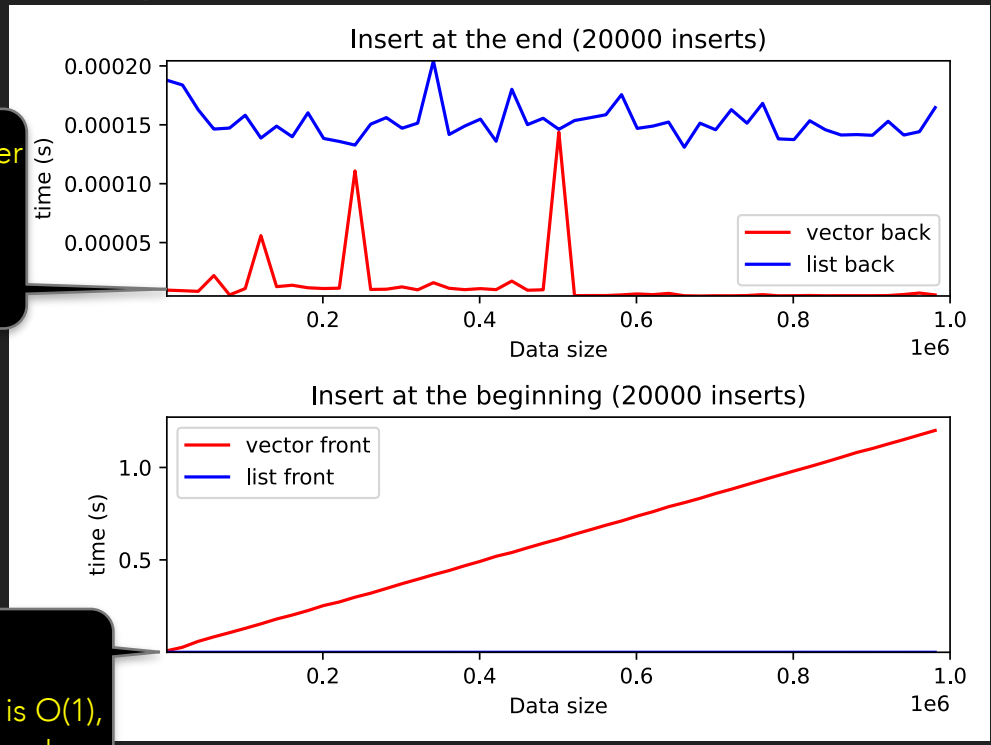



```
node* tmp = new node;
tmp->val = x;
tmp->next = p->next;
p->next = tmp;
```

List vs. Array Insertion

```
std::vector<int> v1, v2;
std::list<int> L1, L2;
...
// in a loop
  ...
  v1.insert(v1.begin(), 1);
  v2.insert(v2.end(), 1);
  L1.push_front(1);
  L2.push_back(1);
...
```

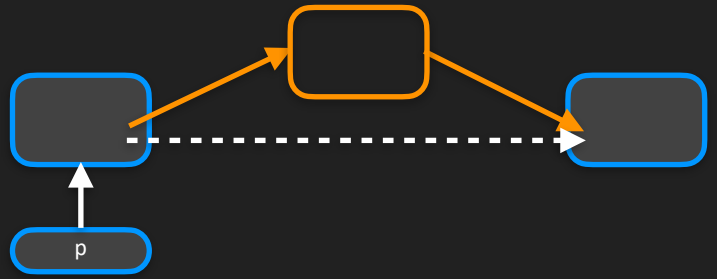
Vector is an order of magnitude faster for push_back



List insertion is O(1), at either end

O(1) Deletions

- In a list, deletions are also O(1)
- Need the link to the node before the one being deleted




```
// To remove p->next
p->next = p->next->next; // OK?
```

- Problem: Memory leak!

```
// The proper way to remove p->next
node* tmp = p->next; p->next = p->next->next; delete tmp;
if (tail==tmp) tail = p;  // To be strictly safe, do this before delete
```

Data-Structures & Algorithms

with

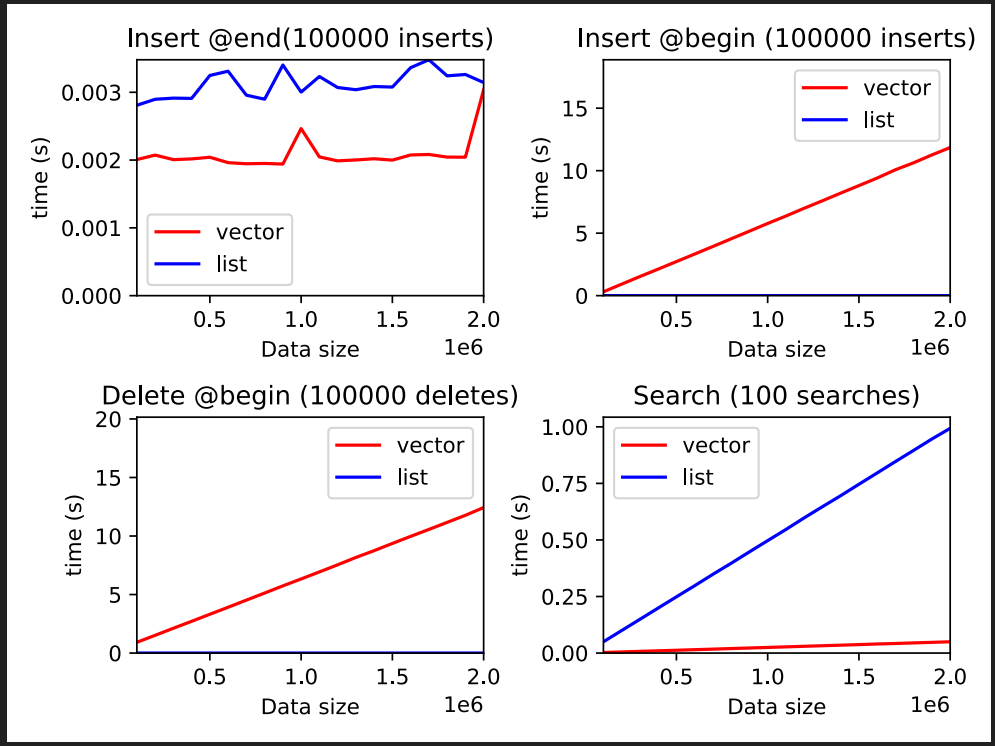
Manoj Prabhakaran

Lecture 4

Linked Lists: Doubly-Linked Lists

Recap: Lists vs. Arrays

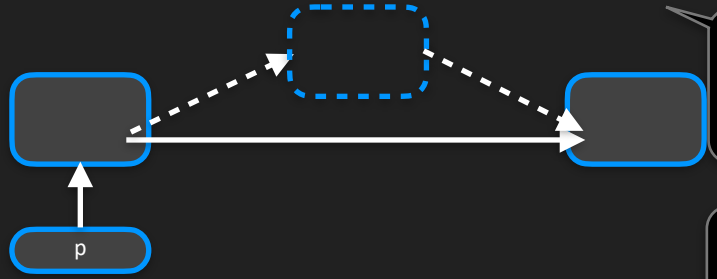
- Two simple data structures with Θ(n) searches
  - Concretely, searches in an array are faster (e.g., by an order of magnitude)
- Lists allow Θ(1) insertion and deletion anywhere in the list, including the beginning, but it is Θ(n) for vectors
  - Aside: insert/delete in vector is much faster than search, due to hardware support of bulk moves



O(1) Insertion and Deletion in a List

- Given a pointer to the node after which the insertion/deletion should be

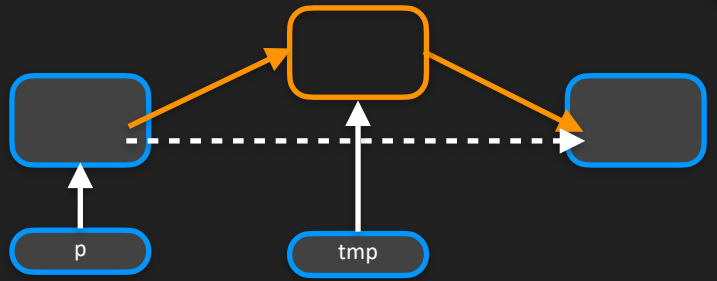
```
// to insert a new node with val x after the node p points to
p->next = new node{x,p->next};
if(p==tail) tail = p->next;
```



Inserting as the head needs slightly different code

```
// to remove p->next, without memory leak
node* tmp = p->next;
p->next = p->next->next;
if (tmp==tail) tail = p;
delete tmp;
```

Deleting the head needs slightly different code



Why Linked Lists

- O(1) insertion and deletion
  - Once we have the location where insertion/deletion happens
  - No existing data nodes are moved during insertion/deletion (except the one deleted)
- Insertion/deletion will not invalidate a node pointer already at hand (unless the node it is pointed to is the one deleted)
  - Can hold search results for a long time, with intervening insertions/deletions (of other nodes)

Why Arrays

- If only insertions at the end needed, arrays are better than lists
  - An important feature of arrays: random access. Directly access the i^{th} element in O(1) time
    - In a list, need to start from the beginning and sequentially proceed to the i^{th} element, in O(n) time
  - Arrays have less space overhead, faster sequential access (no pointer to follow), faster push_back (no memory allocation each time), and guaranteed to be contiguous in physical memory, which allows random access and also leads to better hardware performance ("cache-friendly")

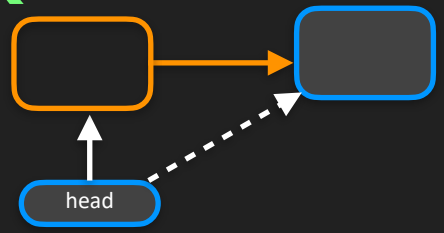
Packaging a List

- To work with a list (in particular to allow push_back and push_front), we need to maintain two pointers, to the front and the back of the list
- Why not make another struct which maintains these two pointers internally?
- E.g., `struct list { node* head; node* tail; /*functions*/};`
- push_back and push_front will automatically update tail/head pointers. User code doesn't have to maintain them (or any pointers)
- When a list object goes out of scope its destructor can deallocate all memory: `list::~list() { while (head) pop_front(); }`

pop_front and pop_back

- Popping from the front of a list:

```
void pop_front() {
  if (!head) return;
  node* tmp = head;
  head = head->next;
  if (tmp==tail) tail = nullptr;
  delete tmp;
}
```



Almost the same code as deleting p->next, but with p->next replaced with head.

- How about pop_back? We need the pointer to the node before tail (so that we can make tail point to it) without O(n) search.
  - Maintain one more pointer in struct list? But how do we update that then?
  - Idea: Every node maintains a pointer to the previous node as well.

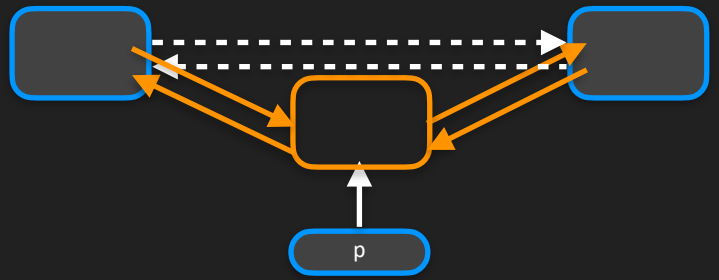
Doubly Linked List

- ```
  struct node {
    int val; node* next = nullptr; node* prev = nullptr;
  };
  ```

- Now, to delete a node, enough to have a pointer to that node itself

```
void list::erase(node* p) { // assumes p is a non-null pointer
  if(p==head) head=p->next; else p->prev->next = p->next;
  if(p==tail) tail=p->prev; else p->next->prev = p->prev;
  delete p;
}
```

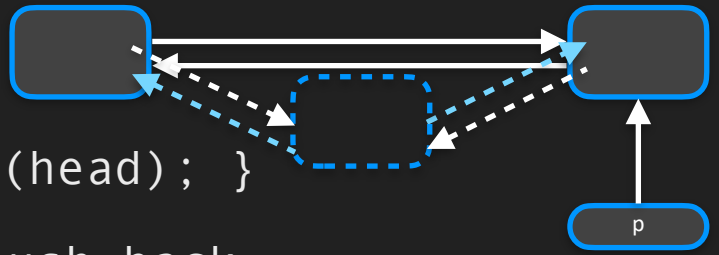
- ```
  void list::pop_front() { erase(head); }
  void list::pop_back()  { erase(tail); }
  ```



Doubly Linked List: Insert

- `void list::insert_before(node* p, int x) { // p non-null`

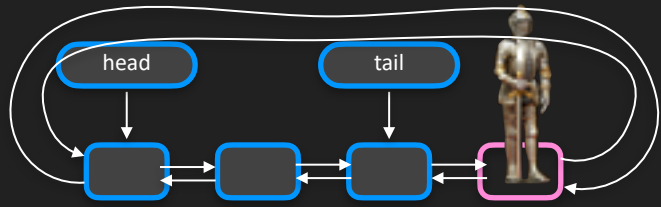
```
  node* tmp = new node{x,p,p->prev};
  if(p==head) head=tmp; else p->prev->next = tmp;
  p->prev = tmp;
}
```



- `void list::push_front() { insert_before(head); }`
- Can symmetrically define `insert_after` and `push_back`
  - Or can do `insert_before(p->next, x)` unless p is `nullptr`
  - Cannot use `insert_before` for `push_back`
- Can avoid special cases using a "sentinel" node (like in std::list)

Sentinel Node

- The code for list operations become cleaner/more uniform if we use a sentinel "end" node, beyond the tail
- The prev field of sentinel will point to the tail. No separate tail pointer.
  - Avoids separate check for inserting at tail (which is now just a standard insert before the sentinel)
- Sentinel's next field can store the head. No separate head pointer.
- head's prev can point to sentinel: avoids special cases for head
- Keep just the sentinel node itself in the list




```
struct list { node sentinel; /*functions*/};
```

Data-Structures & Algorithms

with

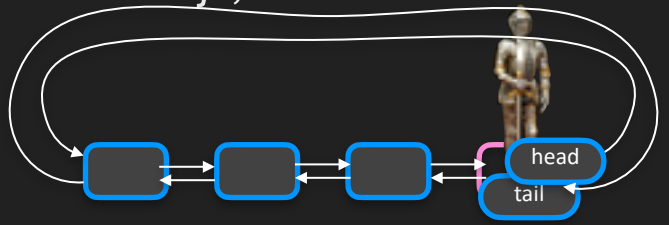
Manoj Prabhakaran

Lecture 5

Linked Lists: Some Cool Tricks!

Recap: Doubly-Linked List w/ Sentinel Node

- `struct node { int val; node *next, *prev; };`
  `struct list { node sentinel; /*functions*/};`
- Insert/Erase code is simplified, and without if-else conditions
  - Per list, one "`val`" field in the sentinel is wasted
  - A significant issue only if a large number of *small* lists are being used



Recap: Doubly-Linked List w/ Sentinel Node

```
struct node { int val; node *next, *prev; };
struct list {
  node sentinel; // sentinel.next is head, sentinel.prev is tail
  list() { sentinel.next = sentinel.prev = &sentinel;} // empty list. no nullptr
  node* begin() { return sentinel.next; } // begin points to head
  node* end() { return &sentinel; }  // end points beyond tail, to the sentinel itself
  void insert(int x, node* p) { // insert before. for insert_after can use insert(x,p->next)
    node* tmp = new node{x,p,p->prev};
    p->prev->next = tmp;  // no special case for p being head
    p->prev = tmp;
  }
  void erase(node* p) {  // assumes p a non-nullptr
    if(p==end()) return; // sentinel cannot be removed
    p->prev->next = p->next;
    p->next->prev = p->prev;
    delete p;
  }
  // Exercise: ~list(), find(), size(), reverse(), join(), ... Add members as needed.
};
```

In std::list

- Very similar to our doubly-linked list with a sentinel. Needn't be circular.
- Doesn't expose a separate node struct. Instead of node* uses a type called `std::list::iterator`
  - Returned by `begin()` and `end()`
  - Supports `*`, `->`, `++`, `--`, `==`, `!=`
    - ⚠️ Effects of `--L.begin()`, `++L.end()` and `*(L.end())` undefined
  - `std::find(L.begin(),L.end(),x)` returns an iterator to where the search ended — a node in the list with value x, or otherwise `L.end()`

`*it` instead of `p->val`. `it->field` instead of `p->val.field`

`++it` and `--it` instead of `p=p->next`, `p=p->prev`

Some Clever Algorithms

- Today
  - An algorithm for efficiently finding a cycle in a (malformed) linked list
  - An algorithm for detecting intersection of two linked lists
  - Lists reversible in O(1) time
- First:
  - Merging two sorted linked lists (a useful subroutine in the mergesort algorithm)

Merging Sorted Lists

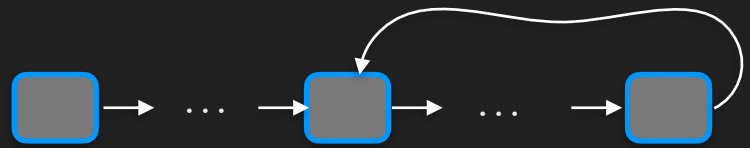
- Problem: Given two sorted (singly) linked lists, L1, L2, merge them into a single sorted list, using O(1) additional space, and without moving any nodes (so that pointers to nodes remain valid after merging)
- Idea: "detach" a node from the front of one of the two lists (whichever has the smaller value) and "attach" it to the back of a merged list

```
struct node { int val; node* next = nullptr; };
struct list {
   node* head=nullptr; node* tail = nullptr;
   bool empty() { return (head==nullptr); }
   node* detach_front(); // doesn't delete detached node
   void attach_back(node* p); // doesn't make a new node
};
```

Exercise

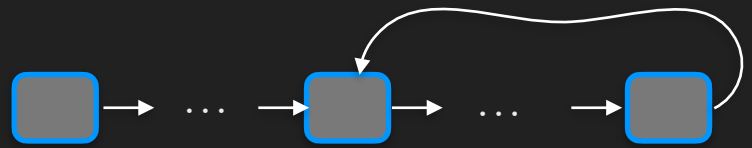
Cycle Detection

- Recall a singly-linked list
- What if, when creating the list, by mistake, a node points to the head?
  - This is a (singly-linked) circular linked list
  - How can you check if this has happened?
    - Easy: Traverse the list until we hit a nullptr or the head again
- A trickier situation: what if a node points to some previous node (not necessarily the head)?
  - Don’t know which node will repeat



Cycle Detection

- With Θ(n) extra space, can solve it easily
  - If each node has space that can be used to "mark" it, then just mark all nodes seen during a traversal. Cycle iff we see a previously marked node.
  - Create a hash table (or set) of all the node addresses encountered, looking for duplicates while adding.
- With just O(1) extra space?
  - In $\Theta(n^2)$ time: At each node, check if its next node is reached first, when starting from the head, through another node: Extra time = 1 + 2 + … + n
- In O(1) extra space and O(n) time?

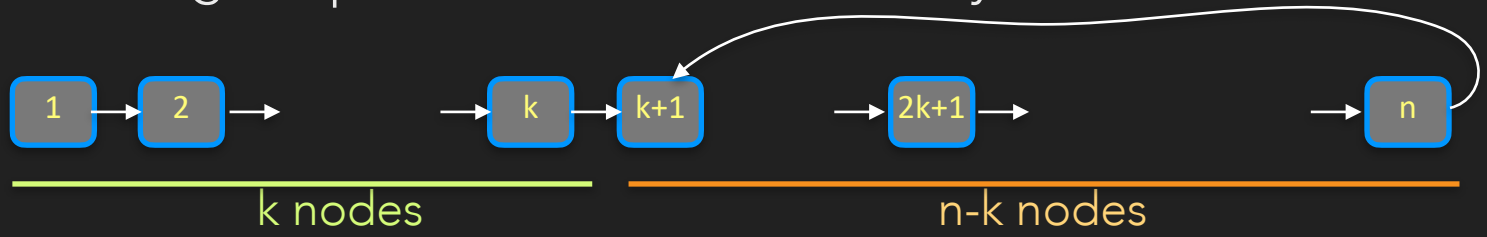


Floyd's Cycle Detection

- ```
  node* slow = head; node* fast = head;
  while(fast && fast->next) {
        slow=slow->next; fast=fast->next->next;
        if(slow == fast) return true; // cycle!
  }
  return false; // no cycle
  ```
  - In a straight list: `slow` will never catch up with `fast`, before `fast` (or `fast->next`) reaches the end of the list and we will return false
  - If there is a cycle: at some point both slow and fast will be in the cycle (because once you enter, you never leave). After that `fast` is chasing and catching up with `slow`, reducing the gap by one at each step. The two will meet!
  ```

Floyd's Cycle Detection

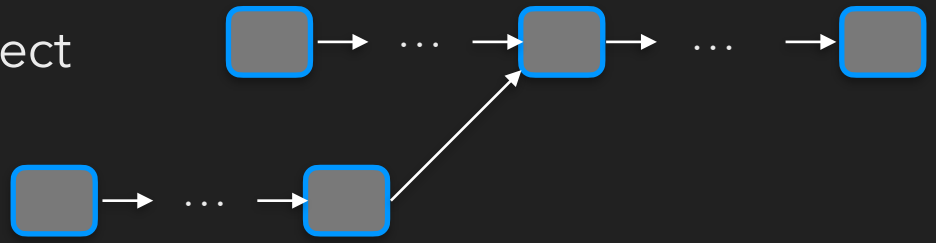
- We can even get a pointer to the start of the cycle!



- For simplicity, let us pretend n >> k.
- In k steps: slow reaches node k+1, fast reaches node 2k+1 (within the cycle)
- In the cycle, slow is (n-2k) steps ahead of fast (slow is at "position n+1")
  - In n-2k more steps, they meet at position n-k+1 (or 2(n-k)+1)
  - Now move one pointer to head, and move both forward in sync till they meet: In k more steps both reach position k+1

Intersection Detection

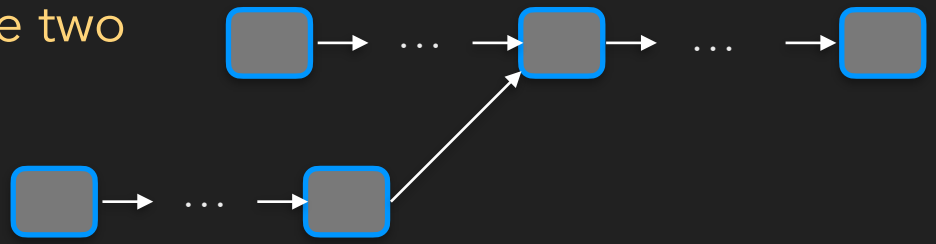
- Suppose two (singly-linked) lists intersect
- To find the first node that belongs to both lists



- Easy using O(n) extra space: Store all the addresses of the nodes of one list in a data structure (set or hash table), and go through each element of the second list checking if it is stored already
- Easy using $O(n^2)$ time and O(1) extra space: For each node in the second list, scan the entire first list to see if it is present there
- Can we do it in O(n) time with O(1) extra space?

Intersection Detection

- Suppose we were guaranteed that the two lists have the same length
- Then, there is some index i such that both lists have their i^{th} node identical



  - Traverse the two lists in synch, checking if both are at the same node
- What if we're not guaranteed same length?
- We can find the lengths of both the lists in O(n) time
- If the lengths are not the same, we can skip the first nodes in the longer list, so that it is left with exactly as many nodes as the shorter one. Now as above.

O(1) Reversal

Challenge: Design it so that it supports O(1) reversal of segments

Hint: For each node, allow either pointer to be next; no need to store this bit, but it can be inferred at traversal time (how?)

How will you redefine O(1) insert?

- Consider designing a doubly-linked list
  - Suppose we want to design it so that it can be quickly reversed
- Idea: store a bit in the list indicating which is the forward direction:

```
struct node { int val; node *A, *B; };
struct list { node S; bool A_is_next; /*..*/};
// if A_is_next, then A is head, else S.B is head
```

  - To reverse the list, simply toggle this bit:

```
void list::reverse() { A_is_next = !A_is_next; }
```

  - In other functions, instead of `p->next`, `p->prev`, use `next(p)`, `prev(p)`:

```
node*& list::next(node* p) { return A_is_next? p->A : p->B; }
node*& list::prev(node* p) { return A_is_next? p->B : p->A; }
```

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 6

Stack

Stack

- What is a stack?
  - Objects (say books) are kept one on top of the other
  - An item can be added only to the top of the stack
  - Can remove only the top-most item (the most recently added)
  - Last-In First-Out (LIFO)
- Why are stacks useful in programs?
  - To support "undo", a stack can used to store actions so far
  - Algorithms for parsing a string according to syntax rules ("grammar") use a stack
  - Systematic searches often use a stack too
  - Memory allocated for function calls naturally use a stack (last function invoked is the first to return)
  - …

A Quick Stack Implementation

```
struct charStack {
    vector<char> stk;
    bool pop(char& x) { // if stack is empty, return false
        if (stk.empty()) return false;
        x = stk.back(); stk.pop_back(); // back is top
        return true;
    }
    void push(char x) {
        stk.push_back(x); // back is top
    }
    void clear() { stk.clear(); }
    bool empty() { return stk.empty(); }
};
```

Example: Palindrome using a Stack

- Check if an input string is a palindrome

```
int main() {
  charStack st;
  char x, y;
  int n; std::cin >> n; // read the length of the string first
  for (int i=0; i<n/2; i++) { std::cin >> x; st.push(x); }
  if (n%2) std::cin >> x; // if n odd, ignore the middle character
  for (int i=0; i<n/2; i++) { // compare stored characters with rest
    std::cin >> x;
    st.pop(y); // guaranteed to succeed since we pushed n/2 items
    if(x != y) { std::cout << "Not a palindrome!" << std::endl; return 1; }
  }
  std::cout << "Palindrome!" << std::endl;
}
```

assuming no input error

Example: Undo Stack

```
bool docommand(char c, charStack& history); // push command into history
bool undo(charStack& history); // pop one command from history & reverse it
int main() {
  turtleSim();  // our commands are for the turtle (from CS101)
  cout << "Enter f/r/l for forward/right/left, u to undo, q to quit: ";
  charStack history; history.clear();
  char input;
  for(cin >> input; !cin.fail(); cin >> input) {
    if(input == 'q')
      break;
    else if (input == 'u')
      if(!undo(history)) // try undoing
        cerr << "Undo stack empty!\n";
    else if (!docommand(input,history)) // try doing
      cerr << "Ignoring unrecognized command" << endl;
  }
}
```

Example: Undo Stack

Demo

```
bool docommand(char c, charStack& history) {
  if (c == 'f' || c == 'r' || c == 'l') {
    history.push(c);
    if (c == 'f') forward(10);
    else if (c == 'r') right(90);
    else /* (c == 'l') */ left(90);
    return true;
  }
  return false;
}
```

```
bool undo(charStack& history) {
  char c;
  if(!history.pop(c)) {
    return false;
  } else {
    if (c == 'f') forward(-10);
    else if (c == 'r') right(-90);
    else /* (c == 'l') */ left(-90);
    return true;
  }
}
```

Vector Problems

- A vector provides more features than a stack
  - In particular, random access
  - But comes at a cost!
    - To keep data contiguous, reallocates memory and moves data, if vector grows beyond the reserved size.
      - Moving data from one vector to another costs O(n) time in the worst-case
- A stack implementation doesn't need to suffer this cost!

Lists for Stacks

- If we had an a priori bound on the stack size, an array/vector is an excellent fit for implementing a stack
  - It supports O(1) adding and deleting items at the end (stack's top), with concrete speeds much faster than a linked list
- The only problem is that the stack may outgrow the array
- Idea: If a fixed-size stack fills up, add a second fixed-size stack on top of it
  - The first stack should be accessible from the one on top (if the top one is fully popped): top stack should point to the one below it
  - A linked list of fixed-size stacks!

A Fixed-Capacity Stack

```
struct stackBlock {
   const int capacity = 512; // fixed capacity
   char stk[capacity]; // will use an array instead of std::vector
   int top = -1; // keep track of last element's index. -1 for empty.
   bool pop(char& x) { // if stack is empty, return false
        if (top < 0) return false;
        x = stk[top--]; // decrement top after retrieving into x
        return true;
    }
    bool try_push(char x) { // if stack already full, return false
        if(top == capacity-1) return false;
        stk[++top] = x; return true; // increment top before storing x
    }
    void clear() { top = -1; }
    bool empty() { return top==-1; }
};
```

A List of Bounded Stacks

```
struct stackNode { stackBlock block; stackNode* below = nullptr; }
struct stackList { // a list of stackNodes
  stackNode* head = nullptr; // top most stackNode
  bool empty() { return head==nullptr; } // stack empty iff list empty
  void push(char x) {
    if(head==nullptr) head = new stackNode; // if adding to empty list
    if(!head->block.try_push(x)) { // try pushing to top stack
      stackNode* tmp = new stackNode; tmp->below = head; head = tmp;
      head->block.try_push(x); // push again after adding a new node
    }
  }
  bool pop(char& x) {
    if (empty()) return false;
    head->block.pop(x); // if not empty, pop from top stack
    if(head->block.empty()) { // if it is empty after popping, delete it
      stackNode* tmp = head->below; delete head; head = tmp;
    }
    return true;
  }
};
```

Exercise: An Issue to Fix

- In this stackList, a series of strictly alternating pushes and pops at the beginning (or at the beginning of a new stackBlock), each push/pop will result in adding/deleting a stockBlock
  - Still O(1) worst-case, but larger constants
- An idea: Add a new block on top when the top block is full (like before); but don't delete the top block when it turns empty -- delete only when the block below becomes empty
  - Allow for the possibility that head is not where the top is
- Exercise

Forgetful Stack

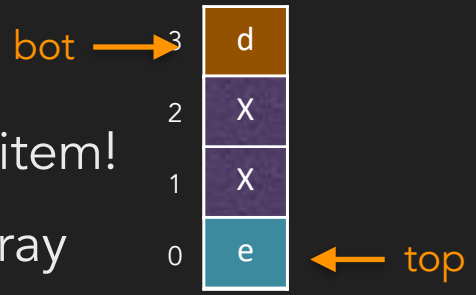
- An unlimited stack may not be suitable for many applications
- E.g., undo stack in an image editor
  - The information required to undo an action could be large (e.g., a snapshot of the image before a lossy blur operation)
  - Storing the undo information for all the commands in an editing session can grow to Giga Bytes of memory.
  - Better to have an a priori bound on the size of the undo stack
- Idea: When pushing a new item into a full stack, we can just forget the oldest item

Forgetful Stack

- How can you efficiently implement a forgetful stack
- Since fixed size, an array suffices: No need for a linked list or std::vector
- But what should we do once the array fills up?
  - Naively: Move all the items in the array to the left (letting the left-most item disappear) and make room for the new item at the right end
    - Problem: Push becomes $\Theta(n)$
  - We shouldn't move the items already in the stack, except possibly the one at the bottom (which is getting forgotten)
  - So we must overwrite the bottom element with the new item!

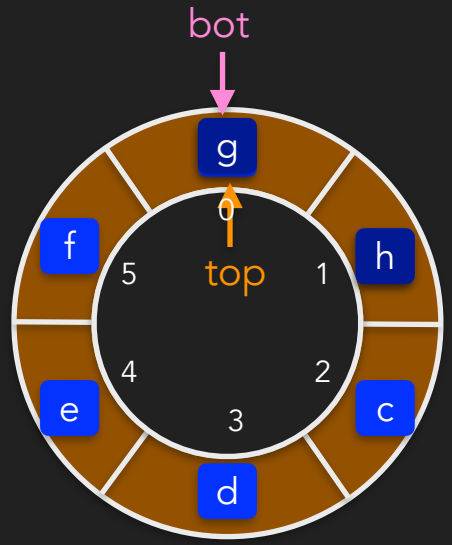
Forgetful Stack

- How can you efficiently implement a forgetful stack
- Since fixed size, an array suffices: No need for a linked list or std::vector
- But what should we do once the array fills up?
  - We must overwrite the bottom element with the new item!
- Top position moves around, modulo N, the size of the array
- Bottom position also moves mod N (only when pushed up by top)
- Stack size can be any of N+1 values (0 through N), but the gap between top and bot can be only one of N values (0 through N-1)
  - Will use a flag to distinguish between two cases (zero or one item)



Forgetful Stack

```
struct stackRing {
  const int N = 512; // capacity
  char array[N];
  int top = 0, bot = 0;
  bool is_empty = true; // for us top==bot can mean 0 or 1 item
  void inc(int& i) { i = (i==N-1 ? 0:i+1); } // increment mod N
  void dec(int& i) { i = (i==0 ? N-1:i-1); } // decrement mod N
  void push(char x) {
    if(is_empty) is_empty = false; // top and bot stay put
    else { inc(top); if(top==bot) inc(bot); }
    array[top] = x;
  }
  bool pop(char& x) {
    if(is_empty) return false;
    x = array[top];
    if(top==bot) is_empty = true; // had one item: don't move top
    else dec(top);
    return true;
  }
};
```



Summary

- A stack provides only push and pop operations: sufficient for many interesting applications
- No random access and hence no need to maintain items contiguously
  - Implementation using a vector is not a good fit
- We saw three implementations with O(1) worst-case push/pop
  - stackBlock: A stack with a pre-determined capacity, using an array
  - stackList: An (uncapped) stack using a list of stackBlocks
  - stackRing: A stack with a pre-determined capacity, that forgets oldest items, using an array

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 7

Queue

Queue

- What is a queue?
  - An item can be added only to the back of the queue
  - Can remove only the front-most item
  - First-In First-Out (FIFO)
- Why are queues useful in programs?
  - Handling events (key presses, mouse clicks, network packets arriving at a router, …) in the order in which they happened
  - Maintaining a window of recent items: Add new item, remove old item
    - A forgetful stack (without any popping) will work too!
  - Systematic searches (We said that about stacks too!)
  - …

Queue

- Can implement using a linked list, and have O(1) enqueue and dequeue operations
- A quick implementation using a vector?
  - Say a fixed-capacity queue?
  - Not immediate! Vector offers O(1) operations at only one end
- Will come back to this soon. For now, let's assume we have a fixed-capacity queue
- Can we implement a full-fledged queue given a fixed-capacity queue?
  - Yes! Similar to what we did for stacks!

A Queue as a List of Bounded Queues

```
struct queueBlock; // TODO
struct queueNode { queueBlock block; queueNode* behind = nullptr; }
struct queueList { // a list of queueNodes
  queueNode *head = nullptr, *tail = nullptr;
  bool empty() { return head==nullptr; } // queue empty iff list empty
  void enqueue(char x) { // will enqueue into tail node
    if(tail==nullptr) head = tail = new queueNode; // if empty list
    if(!tail->block.try_enqueue(x)) { // may fail if tail is full
      queueNode* tmp = new queueNode; tail->behind = tmp; tail = tmp;
      tail->block.try_enqueue(x); // try again after adding a new node
    }
  }
  bool dequeue(char& x) {
    if (empty()) return false;
    head->block.dequeue(x); // if not empty, dequeue from front block
    if(head->block.empty()) { // if it is empty after dequeue, delete it
      queueNode* tmp = head->behind; delete head; head = tmp;
      if(head==nullptr) tail = nullptr;
    }
    return true;
  }
};
```

```
bool front(char& x) {
  if(empty()) return false;
  return head->block.front(x);
}
```

An Application

- A simple moving average is used in many situations to monitor real-time performance of systems
  - E.g., "telemetry data" measure system load at any point. If the load has been high for sometime, need to allocate more resources
  - Don't want to allocate resources just when the telemetry data spikes (the spike may pass)
  - Given telemetry data x_i, use $a_t = \frac{s_t}{N}$ where $s_t = x_t + x_{t-1} + \ldots x_{t-N+1}$
  - Can update from s_{t-1} to s_t in O(1) time: $s_t = s_{t-1} + x_t - x_{t-N}$
    - But need to remember x_{t-N} from time $t - N$ till time t

Simple Moving Average

```
#include <queue>

struct Averager {
  const size_t N = 50; // window (size_t, as returned by std::queue::size())
  std::queue<double> data;
  double sum = 0;
  void ingest(float x) {
    data.push(x); sum += x;
    if(data.size() > N) { sum -= data.front(); data.pop(); }
  }
  bool getAverage(double& a) {
    if(data.size() < N) return false;
    a = sum/N; return true;
  }
};
```

`std::queue` uses `push()`, `pop()` instead of `enqueue()`, `dequeue()`

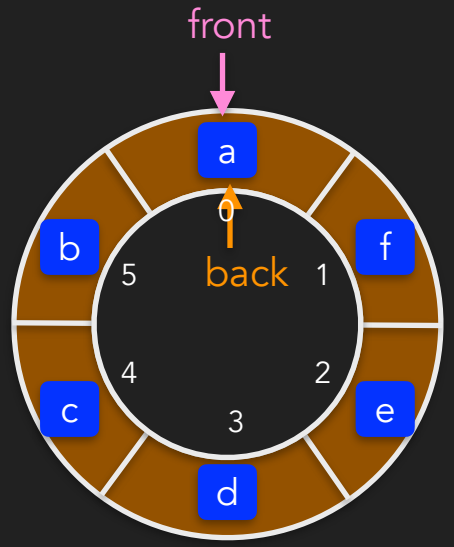
`pop()` doesn't return the dequeued value; use `front()` to read it before popping.

Fixed-Capacity Queue

- Our problem: Vector offers O(1) operations at only one end, but queue needs operations at both ends
- We already solved a very similar problem
  - Recall Forgetful Stack
    - Had to push/pop items at one end
    - But also had to forget from the other end!
  - We implemented it using a circular array (array indexed modulo the size of the array)
  - Same idea works for fixed-capacity queues!

A Fixed-Capacity Queue

```
struct queueBlock {
  const int N = 512; char Q[N]; // fixed capacity Q
  int front = 0, back = 0; // Can be any index with front==back
  bool is_empty = true; // front==back can mean 0 or 1 item
  void dec(int& i) { i = (i==0 ? N-1:i-1); } // decrement mod N
  bool full() {return (back==0 && front==N-1)||(back==front+1);}
  bool try_enqueue(char x) {
    if(full()) return false;
    if(is_empty) is_empty = false; else dec(back);
    Q[back] = x; return true;
  }
  bool dequeue(char& x) {
    if(is_empty) return false;
    x = Q[front];
    if(front==back) is_empty = true; // had one item, now empty
    else dec(front); // if not emptying, decrement front
    return true;
  }
  bool front(char& x) { x = Q[front]; return !is_empty; }
};
```



Summary: Implementing a Queue

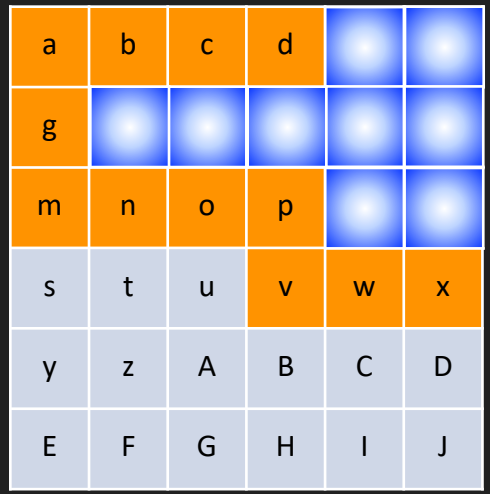
- Fixed-Capacity queue from circular arrays
- Full-fledged queue from a linked list of fixed-capacity queues
  - O(1) inserts and deletes
- The constant hidden in O(1) can be large if every insertion/deletion needs to add/delete a node in the list
  - Recall: Same issue in our stackList implementation last time
  - Mitigated by removing nodes slightly lazily, so that adding/removing nodes will be required only once every B operations (or more), B being queueBlock capacity
    - Same fix applies here too

Another Application: Systematic Exploration

- In the game of minesweeper, some cells have mines under them. Other cells either have a neighbouring cell that is mined (dangerous cells) or not (safe cells).
- When you click on a safe cell, it opens up an “island” of safe cells surrounded by a sea of dangerous cells
- How do you find the cells belonging to this island?
- High-level idea: Systematically explore starting from the clicked cells and moving to its safe neighbours
  - Concretely: Maintain a queue of safe cells encountered so far, and checkout the neighbours of each cell in the queue

Another Application: Systematic Exploration

- *If the clicked cell is safe, start by enqueuing it*
- *Repeat while queue not empty:*
  - *Dequeue a cell*
  - *Enqueue all its neighbours that are safe but were not already enqueued*
- Claim: Cells enqueued are exactly those in the island (i.e., connected to the starting node by a "path" of safe cells)



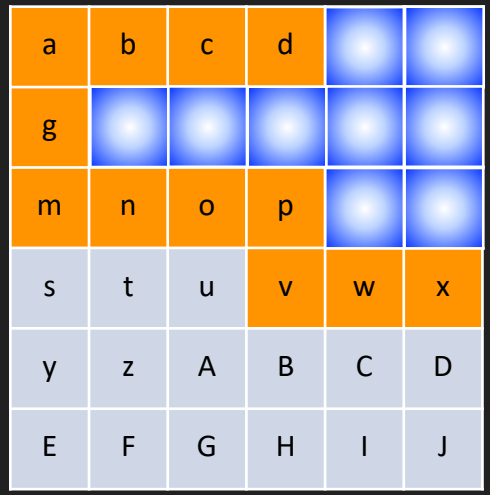
Can you *prove* this?
Hint: Use induction on the length of the path

Later

[f] ➡ [e k l] ➡ [q r e k] ➡ [j q r e] ➡ [j q r] ➡ [j q] ➡ [j] ➡ [i] ➡ [h] ➡ []

Another Application: Systematic Exploration

- What if we had used a stack or any other container?
- *If the clicked cell is safe, insert it into the container*
- *Repeat while container not empty:*
  - *Remove a cell from the container*
  - *Insert all its neighbours that are safe but were not already inserted*
- Still works! But the order of discovery can be different
  - With queue, cells closer to the start are discovered before the further off ones



[f] ➡ [l k e] ➡ [l k j] ➡ [l k q i] ➡ [l k q h] ➡ [l k q] ➡ [l k r] ➡ [l k] ➡ [l] ➡ []

Summary

- A queue provides only enqueue (push) and dequeue (pop) operations
- No random access and hence no need to maintain items contiguously
- Needs O(1) operations at both ends: Can't use std::vector
- We saw two implementations with O(1) worst-case push/pop
  - queueBlock: A queue with a pre-determined capacity, using a circular array
  - queueList: An (uncapped) queue using a list of queueBlocks
- Applications discussed: Processing items in the order they arrived, calculating Simple Moving Averages, systematic exploration.

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 8

Double-Ended Queues

And Amortisation

Deque

A feature of std::deque, which makes it more versatile

- What is a deque?
  - Can add items to either end of a deque (`push_back`, `push_front`)
  - Can remove from either end too (`pop_back`, `pop_front`)
  - Can read from either end (`front()`, `back()`)
  - Random access: Can read the i^{th} item (0^{th} being front) (using `[i]` or `at(i)`)
- Why are deques useful in programs?
  - Used in C++ STL to implement stacks and queues! A deque has all the methods needed by either (and also by vectors)
  - Deque can implement a forgetful stack. Or more custom versions where forgetting depends on the element (not based on the capacity)
  - Systematic searches using a container to hold items to explore, allowing more flexibility, still with O(1) time insertions/deletions

Deque

- Can implement using a linked list, and have O(1) {push/pop}_{front/back}
  - Can we get better constants?
- As a list of fixed-capacity deques (dequeBlock)?
  - A circular array can give a dequeBlock (easy extension of our queueBlock from the last time)
- Almost works, but doesn't support random access!
- First idea (we will improve on this): Reallocate the dequeBlock with double the capacity, each time we outgrow it
  - O(N) worst-case but O(1) *amortised* cost

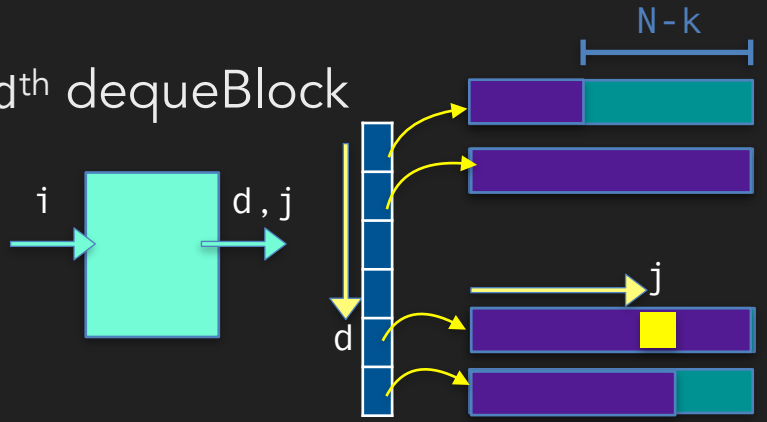
Amortised Cost

Important that we increase the size by a multiplicative factor

- Let us consider the reallocation cost of std::vector (same holds for our dequeBlock)
- Reallocation strategy: On reaching size M, we will reallocate the vector to have size 2M, and copy the M data items to the new location
  - We paid a reallocation cost of M after at least M/2 inserts since the last reallocation
  - Account this cost as spread across those M/2 inserts: each insert is charged an additional cost of 2 = O(1) moves
    - Important: An insert is charged only once — for the next reallocation
  - On average, the cost is still O(1)!

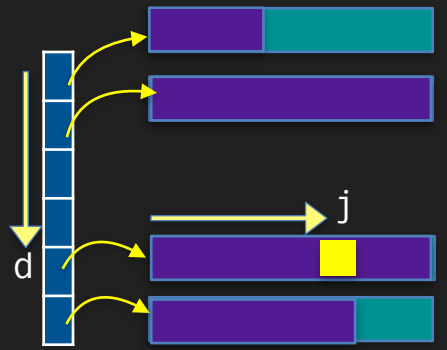
Deque

- For real-time systems, amortised guarantees are often not good enough
- Can we mitigate the O(N) worst-case cost? Maybe reduce the constant?
- A list of dequeBlocks almost worked, but didn't support random access
- What do we need for random access?
  - Translate an index `i` to `(d,j)` such that the item is at index `j` in the d^{th} dequeBlock
- Use a "directory" to get a pointer to the d^{th} dequeBlock
- If the first dequeBlock has `k` elements, `d = (i+N-k)/N`, and `j = (i+N-k)%N` (unless `i<k`, in which case `j=i`)



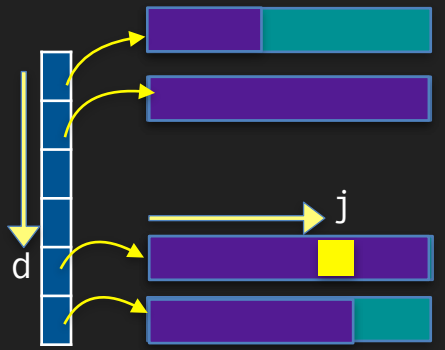
Deque

- directory[d] should have the pointer to the d^{th} dequeBlock
- Should allow adding/removing dequeBlock entries to either end of the directory
  - We need a deque for the directory! But we are trying to build one…
  - Use a dequeBlock? But it will fill up…
    - We will reallocate when that happens! Not very expensive now!
    - Recall from our std::vector analysis: We spend O(M) time for every M entries in the directory
      - That is once for N = M×B items in the deque
    - O(1/B) amortised, O(N/B) worst-case cost



Deque: Exercise

- std::deque (just like std::vector) allows inserting and deleting from anywhere, in O(N) time
- We can actually guarantee O(B) + O(N/B) time
  - Which is technically still O(N) if you consider B a constant. But it is a large constant in practice, and can be modelled as a separate growing parameter.
- Idea: To insert/erase within a dequeBlock, we take time O(B). Then in a sequence of pop/push operations, update the O(N/B) dequeBlocks to one side of the index that is erased/inserted
- Exercise: do it!



An Application: Max in Window

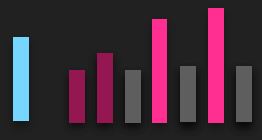
- Finding the max in a window in O(1) time
- Given a streaming sequence of numbers x_i, at any time t, find

  $m_t = \max\{x_t, x_{t-1}, \ldots, x_{t-N+1}\}$

  - Need to store many items from the stream: an x_i that is not the maximum in the current window can later become the maximum in a future window (after the current maximum drops out)
- Naive solution: Maintain the N-long window as a queue, and each time scan it to find the maximum. Total time after T inputs: O(NT)
  - Can we do it in time O(T) instead?

An Application: Max in Window

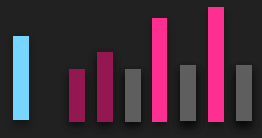
- Idea: Maintain the set of elements within the window that are eligible to be selected as max in the future



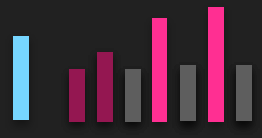
  - When a new element x_t arrives, all the elements in the set which are $\leq x_t$ can be removed
    - Because they will never get a chance to be the max again as this element will be in the window longer than they are
    - Note: Some elements would have been removed by previous items
- Refined idea: Maintain it as a stack of monotonically decreasing elements
  - When an item arrives, the elements less than it are at the top of the stack; pop them and then push the new item

An Application: Max in Window

- Max element at any point is at the bottom of the stack
  - Will use a *deque* to access it
- How about items which get older than the window?
  - The queue is also sorted by arrival time
  - Remove items outside the window from the other end
    - Supported in a deque



An Application: Max in Window



```
#include <deque>

struct item {int val; unsigned int time; };
struct winMax {
  unsigned int now = 0; const int N=50;
  std::deque<item> window; // max candidates from last N items
  void ingest(int x) {
    // first remove at most one item that falls outside the window
    if(now>0 && now - window.front().time == N) window.pop_front();
    item xi{x,now++}; // prepare the new item, and increment time
    // remove all items superseded by the current item
    while(!window.empty() && window.back().val < x) window.pop_back();
    window.push_back(xi); // window always holds the newest item
  }
  bool getMax(item& m) { // window is sorted, max is always at the front
    if(now==0) return false; m = window.front(); return true;
  }
};
```

An Application: Max in Window

- How much time does this take?
- When an item is pushed into the stack, need to pop all smaller items, and this can take Θ(N) time in the worst-case
  - Since T such items, O(NT)
  - That is no better than our naive approach!
- But wait, we can count differently!
  - Each item is pushed in exactly once and popped out at most once (from either end of the deque). Θ(T) push/pop operations in all!
- Processing a single item can take Θ(N) time in the worst case, but that won't happen for every item. Total is O(T).

Amortised O(1) time per item

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 9

Stacks in Action

Course Logistics

- Quiz 1 (CS213 and CS293) dates announced
  - CS293: Tuesday Aug 25, during lab hours
  - CS213: Thursday Aug 27, lecture hour (rooms TBA)
- Partial-credits policy for Weekly-Lab Graded Submission announced
  - You have only two opportunities for claiming partial credits. Use wisely.
- CS213 Participation points: starting next week
  - Bring your device (laptop/phone)

Stack

Recall

- What is a stack?
  - Objects (say books) are kept one on top of the other
  - An item can be added only to the top of the stack
  - Can remove only the top-most item (the most recently added)
  - Last-In First-Out (LIFO)
- Why are stacks useful in programs?
  - To support "undo", a stack can used to store actions so far
  - Algorithms for parsing a string according to syntax rules ("grammar") use a stack
  - Systematic searches often use a stack too
  - Memory allocated for function calls naturally use a stack (last function invoked is the first to return)
  - …

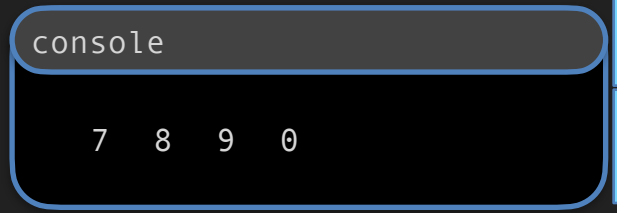
Today: Some More Examples

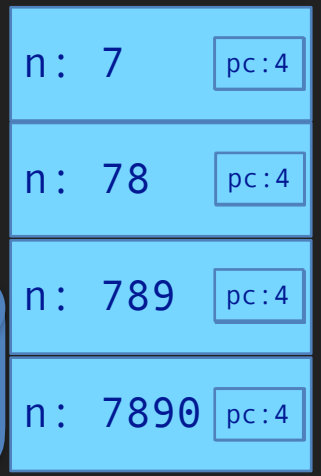
- Implicit stacks in recursive functions
- Balanced Brackets
- A Reverse Polish Notation (RPN) calculator
- An application of monotonic stack

Implicit Stack in Recursion

- When a function is called, memory for its local variables are allocated on a stack
- When the function returns, the stack is popped
- Consider calling the following function with input 7890

```
1. void printdigits(unsigned int n) {
2.  if(n>=10) { printdigits(n/10); }
3.  std::cout << " " << n%10;
4. }
```





Implicit Stack in Recursion

```
void printdigits_stack(unsigned int n) {
  std::stack<unsigned int> s; s.push(n);
  while(n>=10) { s.push(n/=10); }
  while (!s.empty()) { std::cout << " " << s.top()%10; s.pop(); }
}
```

```
1. void printdigits(unsigned int n) {
2.  if(n>=10) { printdigits(n/10); }
3.  std::cout << " " << n%10;
4. }
```

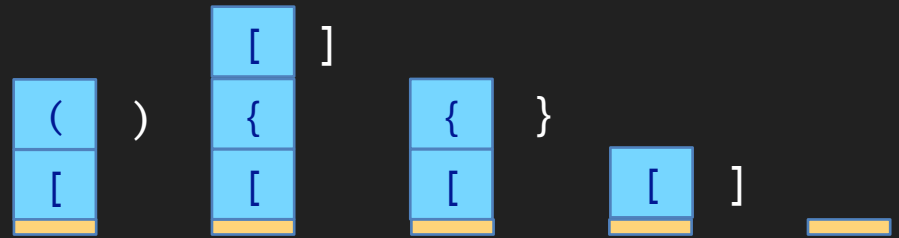
Balanced Brackets

- Consider a string with three kinds of brackets: `()`, `{}` and `[]`
- In a valid string, they should be properly nested: e.g., `[(])` is not OK
  - What is valid?
  - Has the form empty-string, `[`*valid*`]`, `(`*valid*`)`, `{`*valid*`}`, or *valid valid*
    - Either breaks into two shorter (non-empty) valid strings
    - Or the first character should be an opening bracket, and it should be closed/matched by the last one; after stripping off those two, the remaining should be a valid string (of shorter length)
  - How do we check if a string is valid: looking at the first/last characters can't tell which form it is: e.g., `[(){}]` vs. `[](){}`

Balanced Brackets

- Check using a stack, scanning the string left-to-right once
- Each opening bracket is pushed into the stack
- When a closing bracket encountered, pop the stack to see if it matches
- While popping, if there is a mismatch or if the stack empty, unbalanced
- If the string ends while the stack is not empty, unbalanced
- Otherwise (the string ends with no mismatch, and the stack is empty at that point) balanced

[( ) { [ ] }]



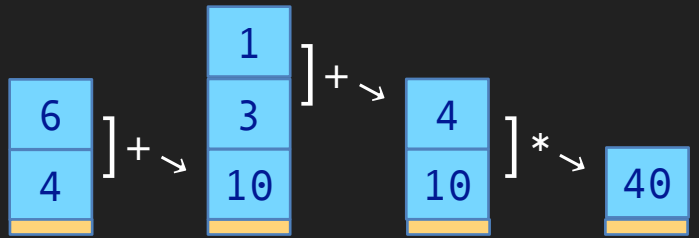
Balanced Brackets

- Claim: Exactly balanced bracket strings are accepted as balanced by this program
- Proof: By induction on the length of the string.
  - Base case: length 0 string. It is balanced, and our program accepts it as balanced ✅
  - Induction: For any k>0, suppose it holds for all strings of length < k. For a k-long string:
    - If valid, will accept: If of the form A B, where each is valid, non-empty, then by induction hypothesis, our stack will be empty after scanning A successfully, and then will be empty again after scanning B successfully. ✅ If of the form {A}, first we push { and then run the program on A, which by induction hypothesis, will return it to {, having never popped it; then on seeing }, will pop it. ✅
    - If accepts, then valid: If we accept, first character is one of [, (or {. Two cases: stack ever became empty before the end, or not. If it became empty after scanning a prefix A of a string AB, then it again became empty after scanning the suffix B; ie it accepts A and B each, and by induction hypothesis, both are valid. Then, by definition, AB is also valid. ✅ If it never became empty till the end, the first opening bracket, say {, stayed at the bottom of the stack throughout and was popped at the end by a matching closing bracket. So the string is of the form {A}; further, if run on A, the program will accept (all conditions met — check!). So A is valid by induction hypothesis. ✅

RPN Calculator

- Standard mathematical expressions need brackets
  - `(a+b)*(c+d)`
- In Reverse Polish Notation (RPN), no brackets needed
- In RPN, also known as postfix notation, the operator is placed after the operands (`arg1 arg2 op` instead of `arg1 op arg2`)
  - Instead of `(a+b)*(c+d)` we have `a b + c d + *`
- Makes it very easy to evaluate an expression in an input stream, using a stack of numbers

```
4 6 + 3 1 + *
```

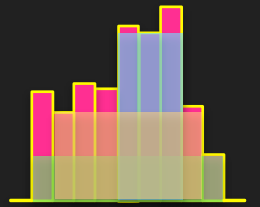


RPN Calculator

```
enum token_t { plus, minus, star, slash, num };
struct token { token_t ty;  double val; };
bool rpnCalc(const std::vector<token>& tokens, double& result) {
  std::stack<double> s;
  for(const token& t: tokens) { // iterate through all the tokens
    if (t.ty == num) { s.push(t.val); } // numbers go into the stack
    else { // apply operator on top two items on stack
      if (s.size() < 2) return false; // stack underflow!
      double b = s.top(); s.pop();
      double a = s.top(); s.pop();
      switch(t.ty) {
        case plus:  s.push(a + b); break;
        case minus: s.push(a - b); break;
        case star:  s.push(a * b); break;
        case slash: if (b == 0) return false; s.push(a / b); break; //divide by 0 is error
        default: return false; // won't get here (unless the code missed a token_t)
      }
    }
  }
  if (s.size() != 1) return false; // correct expressions leave a single value on stack
  result = s.top(); return true;
}
```

Largest Rectangle in a Patchwork of Strips

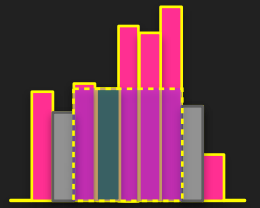
- Equal-width but varying-length strips of cloth are sewn together, with one end aligned, to form a large patchwork
  - What is the largest area of a rectangle aligned with the straight edge, that can be cut-out of this?



- Call a rectangle *maximal* if there is a "limiting strip" whose height (i.e., length) equals the rectangle's height, and the strips to the rectangle's immediate left and right are strictly shorter
- The largest rectangle is maximal, imagining sentinel strips of height 0 to either end
- For each strip we will find the area of the largest maximal rectangle for which that strip is a limiting strip
  - Then the max among those is the largest rectangle

Largest Rectangle in a Patchwork of Strips

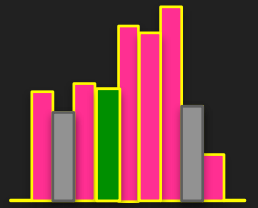
- For each strip we will find the area of the largest maximal rectangle for which that strip is a limiting strip
- We need to find the closest strip to the left and the closest strip to the right that are strictly shorter than this strip



- Naively: scan the strips to the left and to the right, starting from this strip. $\Theta(n)$ time.
  - To do this for all strips, we need $O(n^2)$ time. In fact, $\Theta(n^2)$ time (eg. if all the strips are the same length).
- Goal: $O(n)$ total time

Largest Rectangle in a Patchwork of Strips

- For each strip we will find the area of the largest maximal rectangle for which that strip is a limiting strip
- We need to find the closest strip to the left and the closest strip to the right that are strictly shorter than this strip



- Scanning left to right, push each strip into a stack such that heights are strictly monotonically increasing

Each item is pushed in once and pepped out once

  - When a strip enters the stack we pop entries taller than it. So its closest shorter strip to the left is just under it in the stack
  - When that strip is later popped, it is popped by its closest shorter strip to the right (the ones that didn't pop it are taller)
    - At that point, we know both the neighbours we were looking for

✅